

30.1 A 28nm 127.54TFLOPS/W MXFP6 and 117.42TFLOPS/W MXFP8 Compute-in-Memory Macro with Adaptive-Preserved-Bit-Width and Serial-Dual-Bit-Sliding Schemes

Xing Wang^{1,2}, Yucheng Du¹, Tianhui Jiao¹, Defa Wu¹, Xi Chen¹, Miaoyu Tang¹, Yi Yang¹, Zhichao Liu¹, An Guo¹, Gaoming Fu³, Peng Li³, Jun Dong³, Bo Liu¹, Xinning Liu¹, Weiwei Shan¹, Hao Cai¹, Guangyu Sun⁴, Lin Tong³, Jun Yang^{1,2}, Xin Si¹

¹Southeast University, Nanjing, China, ²National Center of Technology Innovation for EDA, Nanjing, China, ³Xiaomi, Beijing, China, ⁴Peking University, Beijing, China

Abstract

Conventional FP-CIMs suffer from fixed preserved bit-width (PBW), limiting their adaptability and efficiency. This work proposes the first MXFP-CIM macro enabling wide-range adaptive PBW, featuring: (1) A serial dual-bit-sliding scheme; (2) A harmless data mapping scheme

with a hierarchical hidden-bit decoder; (3) An adjustable-PBW MXFP-MAC circuit via twin-stage allocation. The 28nm MXFP-CIM macro achieves a peak energy efficiency of 127.54TFLOPS/W in the MXFP6/6 mode.

With the rapid advancement of artificial intelligence (AI), neural network model sizes have grown exponentially, placing increasing demands on computational bandwidth, memory capacity, and energy efficiency. To alleviate these challenges, low-bit floating-point (FP) formats such as MXFP6 and MXFP8 have emerged as promising alternatives [1,2]. Notably, Deepseek-V3 [3] demonstrated successful FP8-based training and large-scale commercial deployment, highlighting the strong potential of low-bit FP representations. However, conventional FP computing architectures rely on fixed preserved bit-width (PBW), constraining both flexibility and efficiency when adapting to diverse model and layer requirements. This necessitates floating-point compute-in-memory (FP-CIM) solutions that: (1) Support low-bit FP formats, and (2) Offer adjustable PBW.

While existing FP-CIM designs [4-12] have demonstrated impressive performance and energy efficiency, they suffer from three critical challenges (Fig. 30.1.1): (1) Suboptimal arithmetic efficiency in input-alignment schemes [6,7,9,11] due to non-vertical product truncation, which wastes compute resources; (2) Memory capacity underutilization during reconfiguration from variable mantissa/exponent bit-widths, as fixed-precision architectures cannot adapt to dynamic data requirements; (3) Limited variable PBW support due to hardware/software constraints, restricting flexibility for diverse workloads. This work proposes the first MXFP-CIM macro enabling wide-range adaptive PBW, featuring: (1) A serial dual-bit-sliding (SDBS) scheme that enhances FP multiply-accumulate (MAC) computation efficiency by eliminating redundant operations; (2) A harmless data mapping (HDM) scheme with a hierarchical hidden-bit decoder for MXFP6/8, which fully utilizes memory capacity by dynamically adjusting to variable bit-widths; (3) An adjustable-PBW MXFP-MAC circuit via twin-stage allocation, which balances accuracy and efficiency by adaptively scaling precision for different layers/operations. Fabricated in 28nm CMOS technology, a 54kb (24×18×128b) adaptive-PBW MXFP-CIM macro supports MXFP6 and MXFP8 MAC operations for the first time. In the MXFP6/6 mode, it achieves a peak energy efficiency of 127.54TFLOPS/W and a peak area efficiency of 4.44TFLOPS/mm², setting a new benchmark for adaptive-precision FP-CIM designs.

Figure 30.1.2 illustrates the overall architecture, configuration table, and computation flow of the proposed MXFP-CIM macro. The design supports three MXFP6/8 data-type combinations with 3b to 17/23/27b wide-range adaptive preserved bit-width, enabling optimization for workloads spanning efficient inference (low PBW) to high-precision training (high PBW). The architecture comprises a controller, global FP-decoder, bit-width allocator, shared-scale buffer, accumulator, normalizer, input buffer, 24 FP-CIM banks, and other peripherals. Each bank incorporates an 18-row×128-column split-WL 6T-SRAM array alongside local decoders, an exponent-processing unit (EPU), and a mantissa MAC unit. The proposed SDBS FP-MAC flow slides the mantissa input (M_{IN}) 2 bits/cycle (with sign extension) relative to the mantissa weight (M_W), decomposing the multiplication operation across cycles while maintaining vertical truncation. Parallel accumulation aligns products based on exponent sums (E_{SUM}): a higher E_{SUM} triggers earlier M_{IN} sliding to prioritize critical bits. Critically, PBW scales with computation cycles (2-to-14 cycles), enabling dynamic precision via the bit-width allocator.

Figure 30.1.3 illustrates the SDBS FP-MAC flow and implementations of the DBSU, product-compressor (PC), and adder-tree. In the SDBS FP-MAC flow, the exponent adders (EAs) compute $E_{SUM}[4:0]$, the maximum finder (MF) selects the maximum $E_{SUM}[4:1]$, and the sliding destination tracer (SDT) generates sliding controls (SCs). Guided by $E_{SUM}[0]$ and SC, the dual-bit-sliding unit (DBSU) shifts M_{IN} . PCs then compute $M_{IN} \times M_W$ bit-products and compress them into 4-bit values (CVs), which are accumulated by a channel-wise adder tree and normalized in the accumulator & normalizer (A&N) to yield the final mantissa results. The DBSU employs clock gating, 1b shifter, M_{IN} -storage module, and sliding-result (SR) module (8T-C²MOS dynamic registers for area/energy efficiency). At the initial stage, M_{IN} -storage module and SR module are set to $M_{IN}[4:0]$ and the sign bit ($M_{IN}[4]$), respectively. When SC switches from 0 to 1, CLK is enabled, and values in the registers are shifted by 2b per cycle. The 1b shifter determines the connection between the M_{IN} -storage module and SR module based on the LSB value of E_{SUM} (odd/even parity). For the product-compressor, the design uses only six full-adders (FAs) to compress the bit-products, reducing adder tree input bit-width by 42.9% compared to state-of-the-art input-alignment schemes [6,7,9,11]. This reduction in hardware complexity translates to 35.9% less area

overhead and 40.0% lower power consumption. A waveform is presented to show an example of the SDBS FP-MAC, where its adjustable computation cycle provides circuit-level support for adaptive PBW.

Figure 30.1.4 (top) illustrates the harmless data mapping scheme and the hierarchical hidden-bit decoder. In MXFP8 mode, nine MXFP8 (E4M3) numbers are mapped to each 18-row×4-column sub-array, where even rows store sign bits and 3-bit mantissas, while odd rows store 4b exponents. In MXFP6 (E2M3) mode, twelve MXFP6 numbers are mapped per sub-array, with each odd row storing two MXFP6 exponents to avoid memory waste. To support both normal and subnormal numbers in MXFP formats, a hidden-bit decoder is required. In contrast to existing direct hidden-bit decoders for FP-weights [7], this work proposes a hierarchical decoder comprising a global pre-decoder and 24 local decoders. During weight writing period, the global decoder first pre-decodes the hidden-bit and converts weights from sign-magnitude (SM) format to 2's complement (2C) format ($\{S_W, M_W[3:0]\}$); subsequently, $M_W[3]$ is discarded and $\{S_W, M_W[2:0]\}$ is written to the SRAM array. During computation, each local decoder restores $M_W[3]$ from $\{S_W, M_W[2:0]\}$ and E_W . This approach eliminates the SRAM capacity overhead for storing $M_W[3]$ and reduces local decoder area. Compared to direct decoders [7], the proposed hierarchical decoder achieves 63.1% area reduction.

Figure 30.1.4 (bottom) illustrates the proposed twin-stage allocation scheme, aiming to efficiently leverage adjustable PBW FP-MAC circuits. This scheme searches for the optimal preserved bit-width at both layer and group stages to maximize PPA benefits without sacrificing accuracy. In the layer-wise coarse-grained allocation (CGA), a lookup table construction flow iterates through each layer to build a small-size layer-wise allocation table offline, which can be stored in the bit-width allocator. The layer-wise CGA ultimately achieves 14.8% average PBW reduction without accuracy loss on LLaMA-3.1 8B. To further enhance efficiency, group-wise fine-grained allocation (FGA) is introduced. Under the MXFP format, each group contains 32 numbers with an E8M0 shared scale (SS). Since an accumulation vector comprises multiple groups, larger SS values indicate higher contribution to accumulation results and thus require more CIM computation cycles, whereas smaller SS values permit cycle reduction due to lower significance. Consequently, the FGA dynamically allocates group-wise PBW based on ΔSS , yielding 29.3% average PBW reduction. The proposed MXFP-CIM supports three configurations: (1) fixed preserved bit-width; (2) layer-wise CGA; or (3) layer-wise CGA + group-wise FGA. A bit-width allocator and a configurable A&N are designed to support wide-range variable PBW.

Figure 30.1.5 presents the simulation results of the proposed MXFP-CIM macro. Compared to bit-parallel [8,10] and bit-serial [11] alignment schemes, the proposed bit-sliding alignment scheme with CMOS registers achieves area reductions of 3.85× and 1.53×, and energy reductions of 1.44× and 1.43×, respectively. Owing to the twin-stage allocation scheme, the proposed MXFP-CIM achieves the sweet point of accuracy-performance trade-off: it delivers 1.20× to 1.43× higher throughput than global fixed 19b PBW with negligible accuracy loss on LLaMA-3.1 8B; compared to global fixed 11b PBW, it achieves 1.49× accuracy improvement at comparable throughput. As preserved bit-width increases, mean relative error (MRE) monotonically decreases while PPA overhead grows. This work supports a wide-range adjustable PBW, covering simple/complex inference and training applications. For BERT-base, only 3b-to-9b PBW maintains < 1% accuracy loss; for LLaMA-3.1 8B, 11b-to-17b PBW achieves near-lossless accuracy; 19b-to-27b high-precision range optimally supports training applications, and it slightly outperforms BF16 format when training ResNet-18 on CIFAR-100. Compared to existing FP-MAC schemes, the superiority of this work lies in two aspects: (1) higher energy-area efficiency under identical MRE; (2) wide-range adaptive PBW that efficiently adapts to diverse application tasks.

Figure 30.1.6 presents measured results of the adaptive-PBW MXFP-CIM macro, fabricated using 28nm CMOS technology. In MXFP8/8 mode, the peak energy efficiency and area efficiency are 117.42TFLOPS/W at 0.55V and 4.44TFLOPS/mm² at 0.9V, respectively. The shmoo plot confirms that the MXFP-CIM can operate under a 1.2ns clock cycle at 0.9V. Compared to prior SRAM-based FP-CIM designs [5-11] targeting CNN and Transformer inference, this work achieves a 2.8× to 123× improvement in the figure-of-merit (FoM=average energy efficiency×normalized area efficiency×memory density), while

preserving near-lossless end-to-end model accuracy. In addition, by leveraging the proposed wide-range adaptive PBW scheme, this architecture supports not only complex inference workloads (e.g., LLaMA-3.1 8B) but also training tasks (e.g., ResNet-18), delivering of 49.53TFLOPS/W for inference and 19.11TFLOPS/W for training. Figure 30.1.7 presents the die micrograph and performance summary table.

Acknowledgement:

This work was supported in part by the National Natural Science Foundation of China under Grant 62522403, 92264203, 92464202, 92464302, 6232B2022, and 62204036; in part by the National Science and Technology Major Project under Grant 2022ZD0118902, 2023YFB4405100, 2023YFB4405102; in part by the Key Research and Development Program of Jiangsu Province under Grant BE2023020-1; and in part by the Fundamental Research Funds for the Central Universities.

References:

- [1] B.D. Rouhani et al., "Microscaling Data Formats for Deep Learning," *arXiv preprint*, pp. 1-9, Oct. 2023. <https://doi.org/10.48550/arXiv.2310.10537>
- [2] S.-Y. Liu., "LLM-FP4: 4-Bit Floating-Point Quantized Transformers," *arXiv preprint*, pp. 1-14, Oct. 2023. <https://doi.org/10.48550/arXiv.2310.16836>
- [3] DeepSeek-AI et al., "DeepSeek-V3 Technical Report," *arXiv preprint*, pp. 1-53, Feb. 2025. <https://doi.org/10.48550/arXiv.2412.19437>
- [4] S. Lee et al., "A 1nm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications," *ISSCC*, pp. 176-77, Feb. 2022. <https://doi.org/10.1109/ISSCC42614.2022.9731711>
- [5] A. Guo et al., "A 28nm 64-kb 31.6-TFLOPS/W Digital-Domain Floating-Point-Computing-Unit and Double-Bit 6T-SRAM Computing-in-Memory Macro for Floating-Point CNNs," *ISSCC*, pp. 128-129, Feb. 2023. <https://doi.org/10.1109/ISSCC42615.2023.10067260>
- [6] P.-C. Wu et al., "A 22nm 832Kb Hybrid-Domain Floating-Point SRAM In-Memory-Compute Macro with 16.2-70.2TFLOPS/W for High-Accuracy AI-Edge Devices," *ISSCC*, pp. 126-127, Feb. 2023. <https://doi.org/10.1109/ISSCC42615.2023.10067527>
- [7] W.-S. Khwa et al., "A 16nm 96Kb Integer/Floating-Point Dual-Mode-Gain-Cell-Computing-in-Memory Macro Achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-Edge Devices," *ISSCC*, pp. 568-569, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454447>
- [8] Y. Yuan et al., "A 28nm 72.12TFLOPS/W Hybrid-Domain Outer-Product Based Floating-Point SRAM Computing-in-Memory Macro with Logarithm Bit-Width Residual ADC," *ISSCC*, pp. 576-577, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454313>
- [9] W.-S. Khwa et al., "A 16nm 216kb, 188.4TOPS/W and 133.5TFLOPS/W Microscaling Multi-Mode Gain-Cell CIM Macro Edge-AI Devices," *ISSCC*, pp. 252-253, Feb. 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904606>
- [10] Z. Yue et al., "A 51.6TFLOPS/W Full-Datapath CIM Macro Approaching Sparsity Bound and <2-30 Loss for Compound AI," *ISSCC*, pp. 256-257, Feb. 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904702>
- [11] X. Wang et al., "A 28nm 17.83-to-62.84TFLOPS/W Broadcast-Alignment Floating-Point CIM Macro with Non-Two's-Complement MAC for CNNs and Transformers," *ISSCC*, pp. 254-255, Feb. 2025. <https://doi.org/10.1109/ISSCC49661.2025.10904738>

[12] Y. Wang et al., "A 28nm 83.23TFLOPS/W POSIT-Based Compute-in-Memory Macro for High-Accuracy AI Applications," *ISSCC*, pp. 566-567, Feb. 2024. <https://doi.org/10.1109/ISSCC49657.2024.10454567>

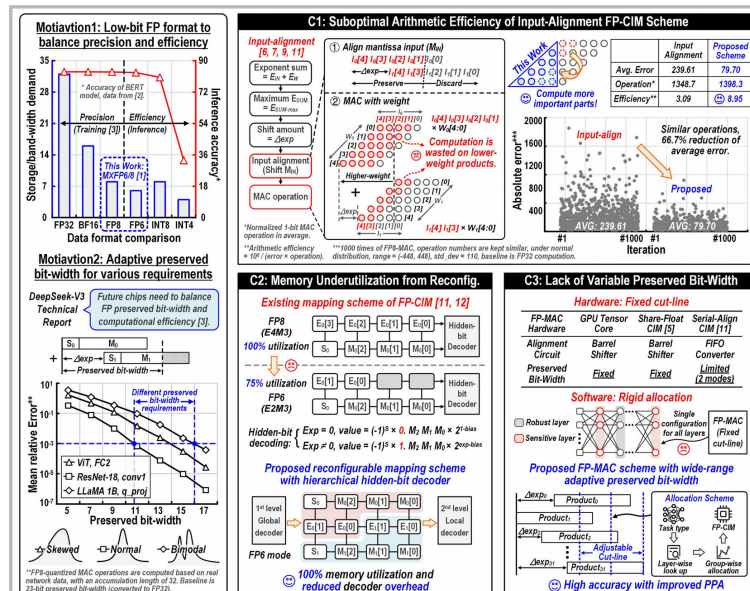


Figure 30.1.1: Motivations, design challenges, and solutions of the proposed MXFP-CIM.

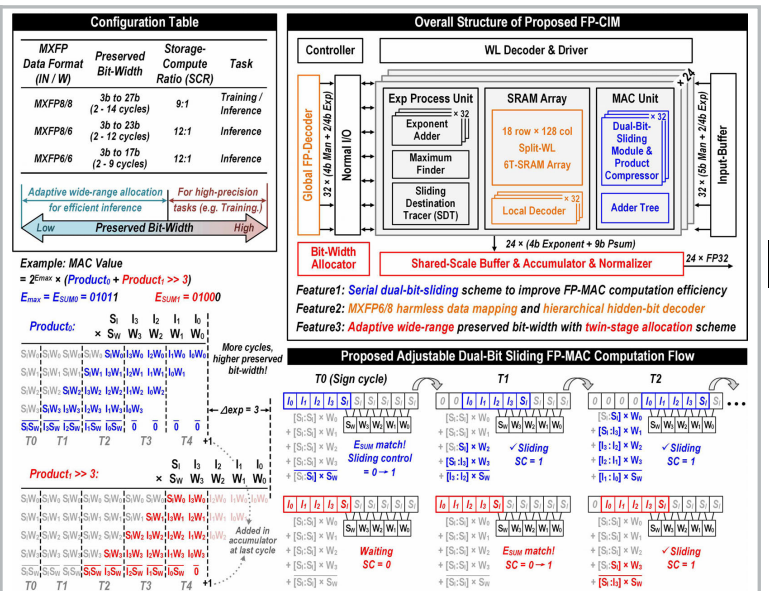


Figure 30.1.2: Configuration table, overall structure and dual-bit sliding computation flow.

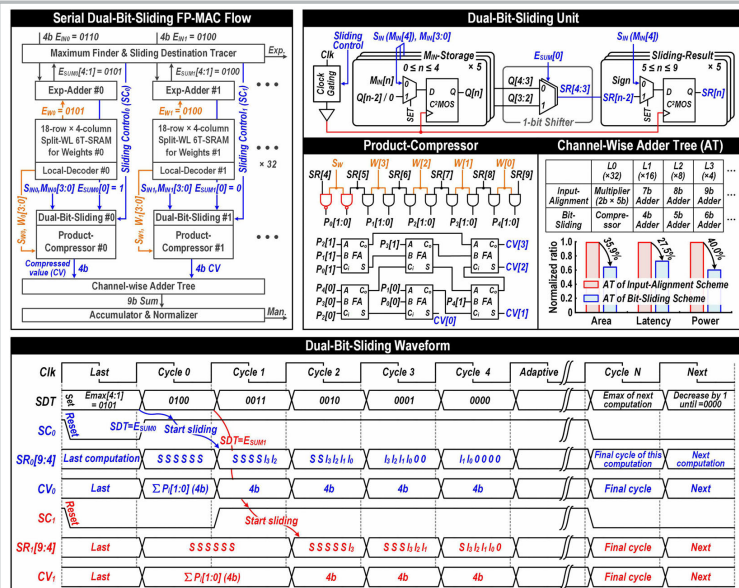


Figure 30.1.3: Details of SDBS FP-MAC flow, implementations of the DBSU, the product compressor, and the channel-wise adder tree.

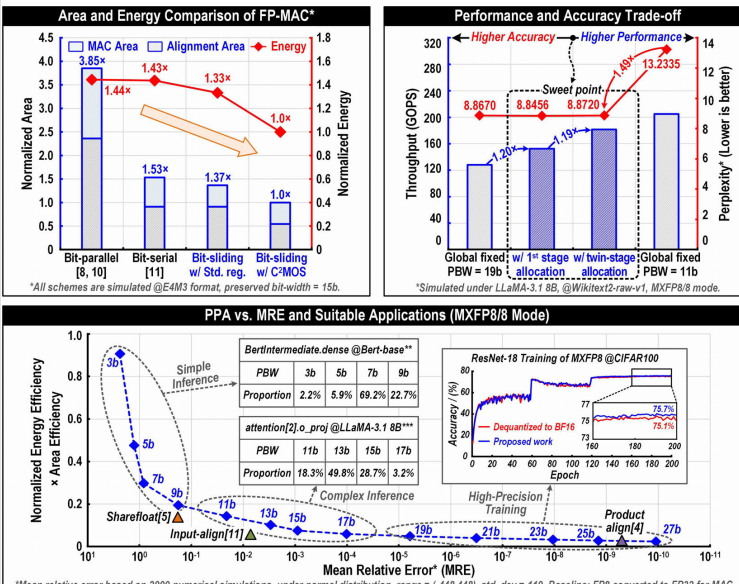


Figure 30.1.5: Simulated performance of the proposed work.

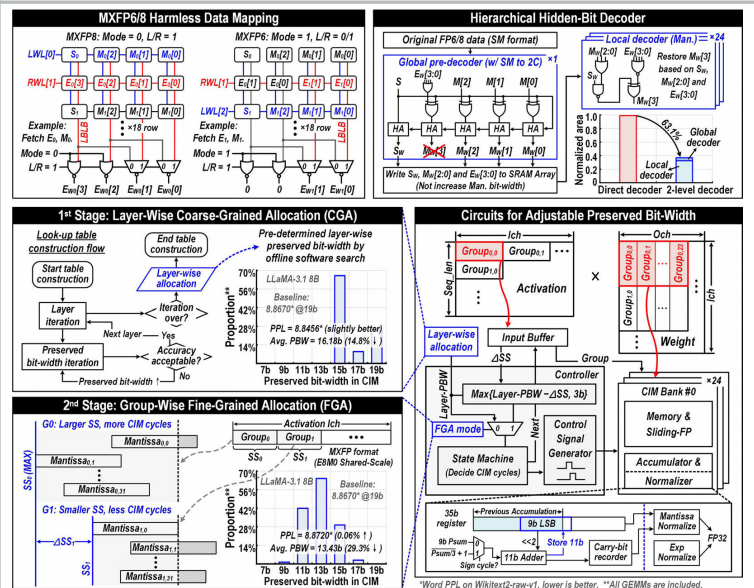


Figure 30.1.4: Mapping scheme for MXFP6 and MXFP8 reconfiguration, hierarchical hidden-bit decoder and layer-group twin-stage allocation scheme.

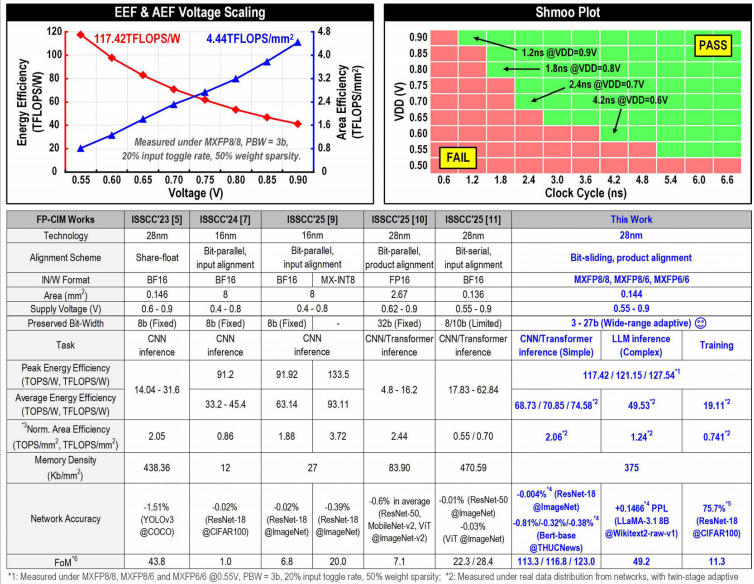


Figure 30.1.6: Measurement results and comparison table.

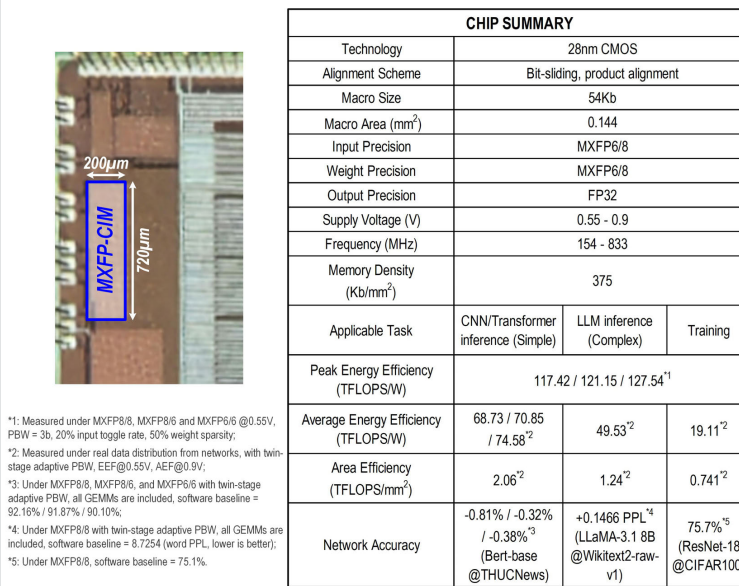


Figure 30.1.7: Die micrograph and chip summary table.